

Balanced Trees

Red Black Trees

1

Red Black Trees

Properties

A **red- black tree** is a **binary search tree** with the following additional properties:

- 1) *Every tree node is either black or red*
- 2) *Root is always black*
- 3) *If node is red then its child must be black*
- 4) *Every path from a node to its descendent leaf has **same number of black nodes** . The number of nodes from root to a leaf is called **black height***

▪It follows that a black node can have either black or red child But a **red node can have only black child**

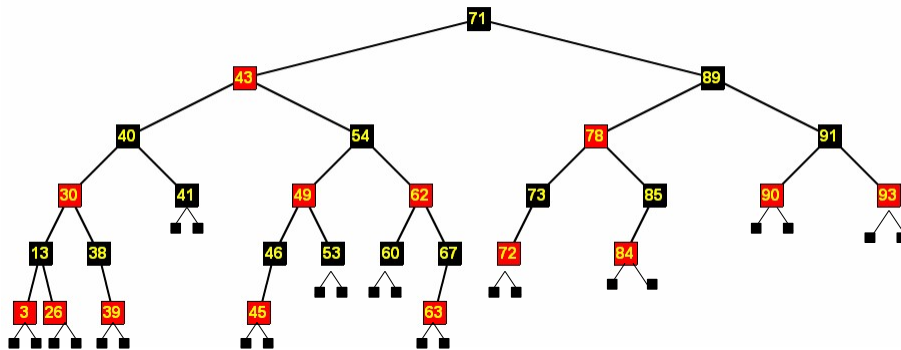
▪By convention the leafs in a red-black child are assumed to be linked to **external black dummy nodes**.

2

Red Black Tree

Structure

A **Red-Black tree** is shown in the figure. The small black rectangles at the bottom are **sentinels or dummy** (Nil) nodes, which do not occupy any storage. By convention these nodes are **black**

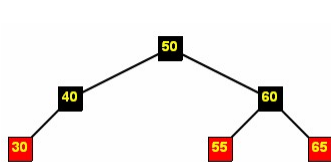


3

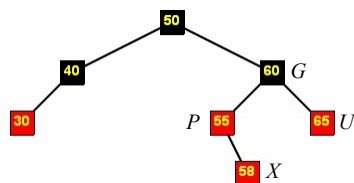
RB Fixing Up

Uncle is Red-Example

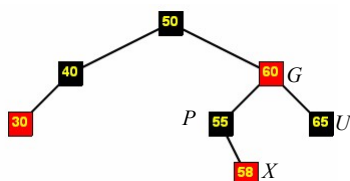
•If **uncle of a given node** is **red**, the RB-violation is fixed up by **re-coloring the parent, uncle and grand parent**. The procedure is illustrated by the following example.



(a) The original tree is **balanced**. It has **black height 2**.



(b) A new **red node X** is added. It causes RB violation. The parent, P, and uncle, U, of X are red. The grand parent, G, is black.



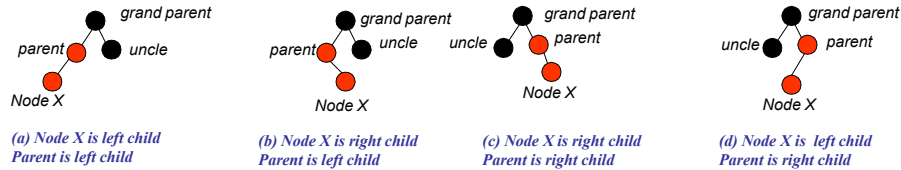
(c) The RB violation is fixed up by re-coloring parent, uncle, and grand parent. The resulting tree is **balanced**. It has **black height 2**.

4

RB Trees Fixing Up

Uncle is Black

When **uncle** of a node is **black**, the node and its parent can occur in four possible configurations with respect to grand parent, as depicted in figures (a)-(d)



- The red-black violation is fixed up by a sequence of **re-coloring** and **rotation operations**. The four configurations are **handled differently**.

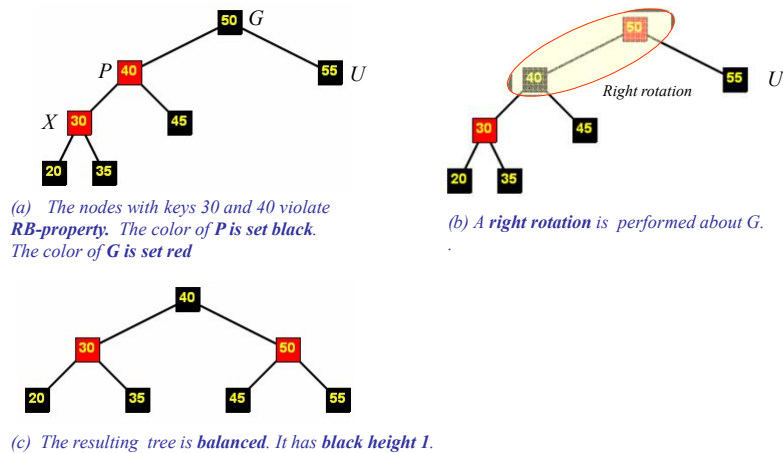
5

RB Fixing Up

Uncle is Black-Case I

- When **uncle is black**, and given **node is left child** and its **parent is left child** of grand parent, the RB fixing up procedure is as follows:

- 1) Re-color **parent to black** and **grand parent to red**
- 2) Perform **right rotation** about the grand parent



6

RB Fixing Up

Uncle is Black-Case II

•When **uncle is black**, and given **node is right child** and its **parent is right child** of grand parent, the RB fixing up procedure is as follows:

- 1) Re-color parent to black and grand parent to red
- 2) Perform left rotation about the grand parent

(a) The nodes with keys 60 and 70 violate RB-property. The color of P is set black. The color of G is set red

(b) A left rotation is performed about G.

(c) The resulting tree is balanced. It has black height 1.

7

RB Fixing Up

Uncle is Black-Case III

•When **uncle is black**, and given **node is right child** and its **parent is left child** of grand parent, the RB fixing up procedure is as follows:

- 1) Perform left rotation about parent
- 2) Re-color new parent to black and grand parent to red
- 3) Perform right rotation about the grand parent

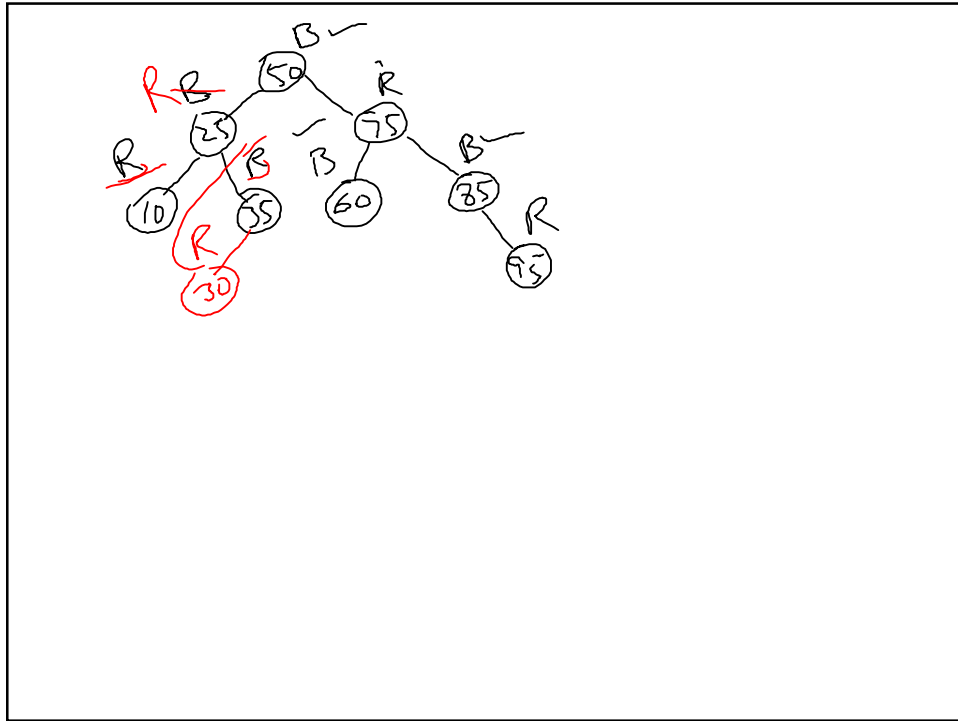
(a) The nodes with keys 40 and 45 violate RB-property. A left rotation is performed about P

(b) The color of new parent is set black. The color of G is set red.

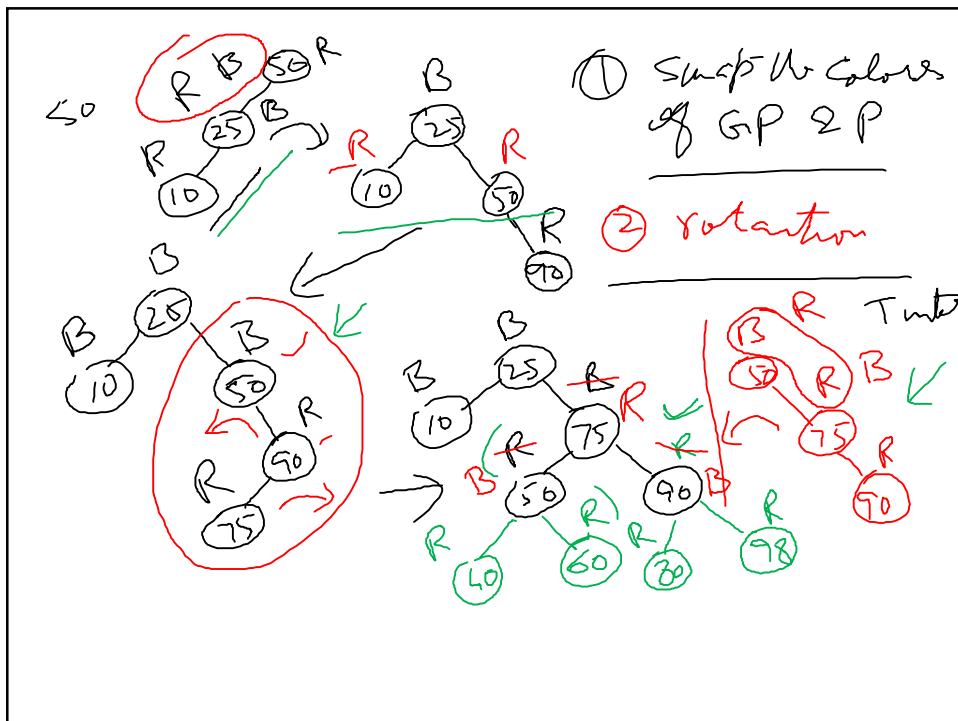
(c) A right rotation is performed about G.

(d) The tree is now balanced

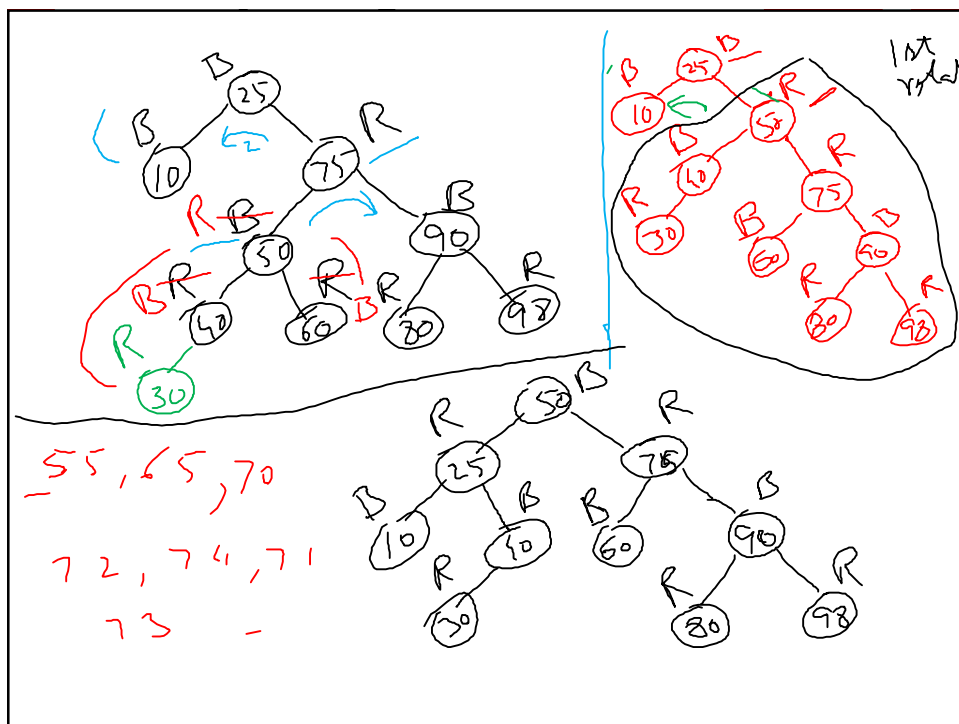
8



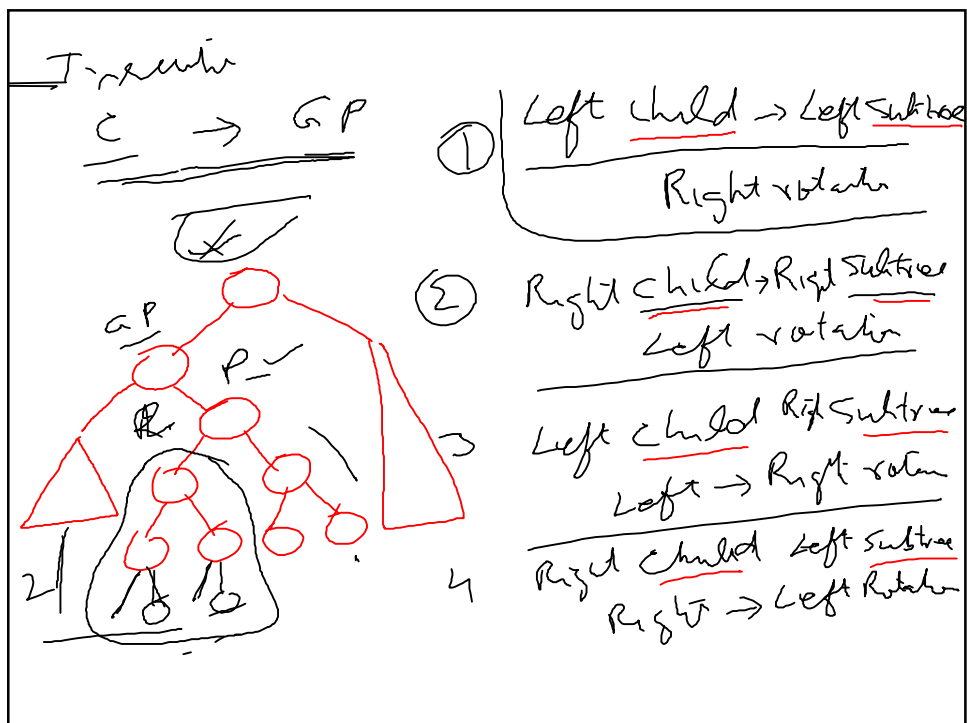
11



12



13

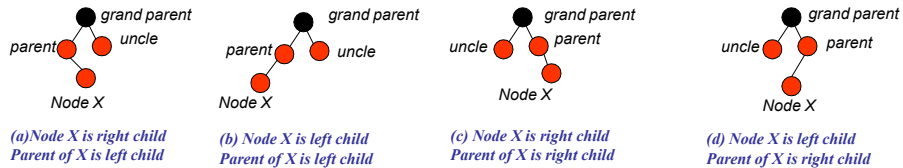


14

RB Tree Fixing Up

Uncle is Red

- When **uncle is red**, a given node *X* and its parent can occur in four possible configurations with **respect to the grand parent**. These are depicted in figures (a)-(d).



- The **RB violation** is fixed up by **re-coloring** of nodes in three steps as follows:
 - 1) Color of **parent** is set **black**
 - 2) Color of **grand parent** is set **red**.
 - 3) Color of **uncle** is set **black**
- The fixing up involves **no structural changes** in the tree.

15

Analysis of Basic Operations on Red Black Tree

16

RB Trees *versus* Unbalanced BST

Comparison of heights

When a set of keys is input in different order, the **RB tree** has comparatively *shorter height* than the corresponding **unbalanced BST**.

- The table below illustrates how the heights of trees vary when a *set of keys 9 identical keys* is stored in different order in an unbalanced BST and in RB tree.

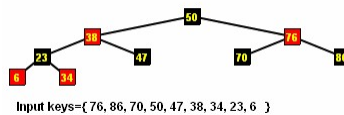
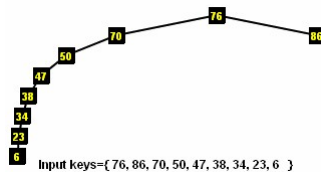
Order of Input keys (from left to right)	# node levels in RB Tree	# node levels in unbalanced BST
47, 23, 86, 6, 34, 70, 38, 76, 50	4	4
50, 23, 86, 6, 34, 70, 38, 76, 47	4	5
50, 47, 70, 76, 38, 86, 34, 23, 6	4	6
70, 76, 86, 50, 47, 38, 34, 23, 6	4	7
76, 86, 70, 50, 47, 38, 34, 23, 6	4	8

17

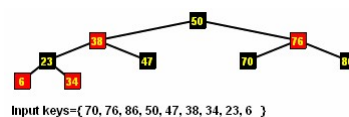
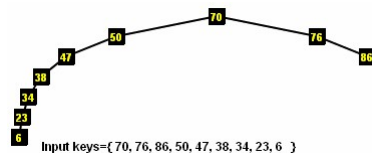
RB Tree versus Unbalanced BST

Comparison-I

An **RB trees has smaller height** than the height of corresponding unbalanced BST for the same set of keys. When BST has **minimum height**, the RB also has the same height. This property is illustrated by the figures. In the example, each tree stores *same set of keys*, which are *input in different order*.



(a) The BST is organized on 8 levels. The corresponding RB tree has 4 levels

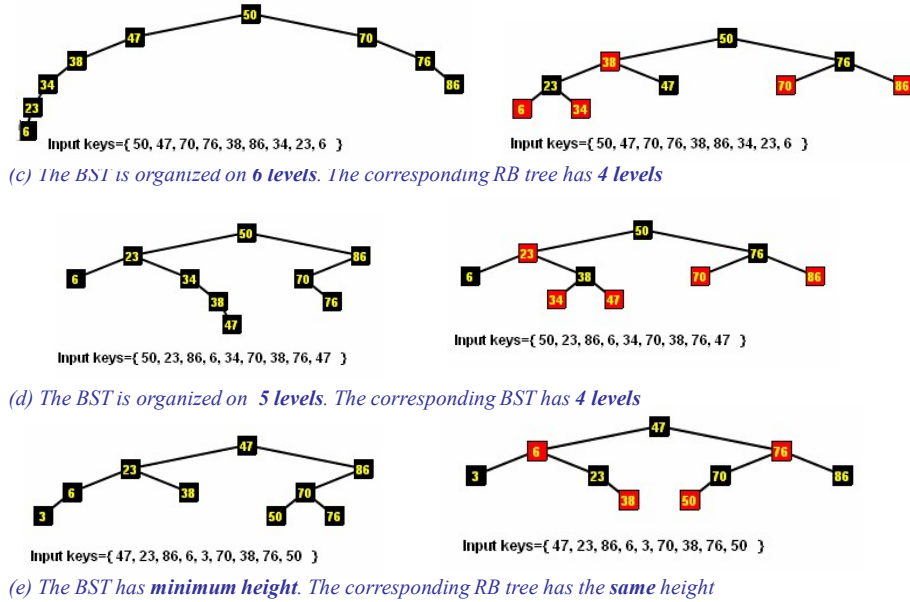


(b) The BST is organized on 7 levels. The corresponding RB tree has 4 levels

18

RB Tree versus Unbalanced BST

Comparison-II



19

Analysis of RB Tree Search

Best and Worst Running times

- The previous theorem establish that if h is the height of an RB Tree storing n nodes then,

$$\lg(n+1) \leq h \leq 2\lg(n+1)$$

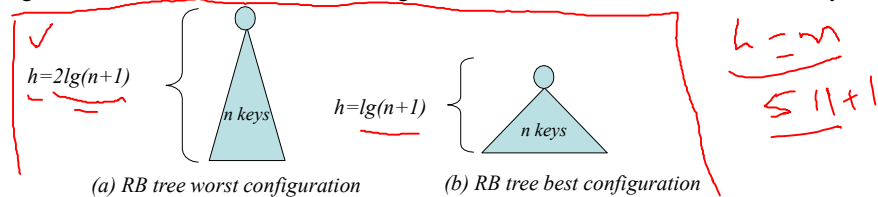
- It follows that an RB tree with n nodes can have a **maximum height** $h_{\max}=2\lg(n+1)$. Thus, the **worst running time for a search operation** is

$$T_{\text{worst}} = O(h_{\max}) = O(2\lg(n+1))$$

- Also, an RB tree with n nodes can have a **minimum height** $h_{\min}=\lg(n+1)$. Thus, the **best running time for a search operation** on an RB tree is

$$T_{\text{best}} = O(h_{\min}) = O(\lg(n+1))$$

- The figures illustrate the best and worst configurations of RB tree with same number of keys.



20